MATH 345 Skeleton Notes

Unit 5: Optimization and training

Section 5.7: Fixed-hidden-layer training

Let $N_{\mathbf{c}}(t) = c_0 + c_1\sigma(t) + c_2\sigma(t-1) + c_3\sigma(t+1)$, where $\sigma(t) = \tanh(t)$.
Only c_0, c_1, c_2, c_3 are trainable.
For example, take data

$$t = (-2, -1, 0, 1, 2), \qquad y = (4, 1, 0, 1, 4).$$

Given data (t_i, y_i), define

$$A = \begin{bmatrix} 1 & \sigma(t_1) & \sigma(t_1 - 1) & \sigma(t_1 + 1) \\ 1 & \sigma(t_2) & \sigma(t_2 - 1) & \sigma(t_2 + 1) \\ \vdots & \vdots & \vdots & \vdots \\ 1 & \sigma(t_m) & \sigma(t_m - 1) & \sigma(t_m + 1) \end{bmatrix}.$$

Training the final layer is the least-squares problem $A\mathbf{c} \approx \mathbf{y}$. If the hidden-layer weights were also trainable, the loss would become nonlinear in the parameters, and gradient descent plus the chain rule would be needed.

Activity: Fixed features, trained coefficients

Let $\sigma(t) = \tanh(t)$, and let

$$N_{\mathbf{c}}(t) = c_0 + c_1\sigma(t) + c_2\sigma(t-1) + c_3\sigma(t+1).$$

For data inputs $t_1, \dots, t_m$, define

$$A = \begin{bmatrix} 1 & \sigma(t_1) & \sigma(t_1 - 1) & \sigma(t_1 + 1) \\ 1 & \sigma(t_2) & \sigma(t_2 - 1) & \sigma(t_2 + 1) \\ \vdots & \vdots & \vdots & \vdots \\ 1 & \sigma(t_m) & \sigma(t_m - 1) & \sigma(t_m + 1) \end{bmatrix}.$$

1. Which entries of A are fixed once the data inputs are known?
2. Which vector is trained?
3. Why is $A\mathbf{c} \approx \mathbf{y}$ a least-squares problem?
4. Why would the problem become nonlinear if the shifts inside σ were trainable?

Tags. `[U5-L06, U5-L07 | C+M+R | Core]`

Note The model is nonlinear as a function of the input t, but linear as a function of the trained coefficient vector $\mathbf{c}$. This is why least squares applies to the fixed-hidden-layer problem.

For the five data points above, this produces a 5×4 design matrix. The lab asks you to build that matrix, solve the least-squares problem for $\mathbf{c}$, and compare the fitted curve with the data.

Note: Machine-learning example: next-token prediction as optimization A language model receives a sequence of tokens and produces scores for possible next tokens. After converting scores into probabilities, training rewards the model for assigning high probability to the

actual next token.
A final hidden vector $\mathbf{h}$ can be converted into scores by an affine rule

$$\ell = W\mathbf{h} + \mathbf{b}.$$

These scores are often called logits. A later nonlinear step converts scores into probabilities, and the loss measures how well the model assigns probability to the correct next token.
For a sequence $t_1, t_2, \ldots, t_L$, a simplified next-token loss is

$$L(\theta) = -\sum_{i=1}^{L-1} \log p_\theta(t_{i+1} \mid t_1, \ldots, t_i).$$

The details of a real language model are complicated, but the optimization idea is familiar: $\theta_{k+1} = \theta_k - \alpha \nabla L(\theta_k)$. The parameters θ include many matrices and bias vectors. Training changes those parameters to reduce the loss. This is why gradients, chain rules, and matrix products are central in modern machine learning.

Activity: Reading a next-token loss as optimization

A simplified language-model training objective can be written

$$L(\theta) = -\sum_{i=1}^{L-1} \log p_\theta(t_{i+1} \mid t_1, \ldots, t_i).$$

A gradient descent step has the form

$$\theta_{k+1} = \theta_k - \alpha \nabla L(\theta_k).$$

1. What are the parameters?
2. What quantity is being minimized?
3. Which symbol is the learning rate?
4. What does the gradient point toward?

5. Why is this an optimization problem, even though we are not studying full model training?

Tags. `[U5-LO2, U5-LO3 | C+M+R | Core]`

Activity: Training Only the Last Layer

Suppose a fixed hidden representation is $\mathbf{h} \in \mathbb{R}^d$, and an output layer computes $\ell = W\mathbf{h}$. For one training example, suppose the loss gradient with respect to the logits is $\mathbf{g} \in \mathbb{R}^m$. Then the gradient with respect to W has the form

$$\nabla_W L = \mathbf{g}\mathbf{h}^T.$$

What is the shape of $\mathbf{g}\mathbf{h}^T$? Why is this a rank-one matrix, unless $\mathbf{g} = \mathbf{0}$ or $\mathbf{h} = \mathbf{0}$? What does the update $W_{\text{new}} = W - \alpha\mathbf{g}\mathbf{h}^T$ change?